

C# Tutorial

A Complete Guide

11 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/c#/>

Getting Started with C#

What is C#?

C# (pronounced "C Sharp") is a modern, object-oriented programming language developed by Microsoft. It runs on the .NET platform and is used for building desktop apps, web apps, games (Unity), mobile apps, and cloud services.

Key Idea: C# is a statically-typed language — variables must have a declared type checked at compile time.

Your First C# Program

```
using System;
namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Basic Syntax

- Statements end with semicolon ;
- Blocks use curly braces { }
- using imports namespaces

- `Main` is the entry point

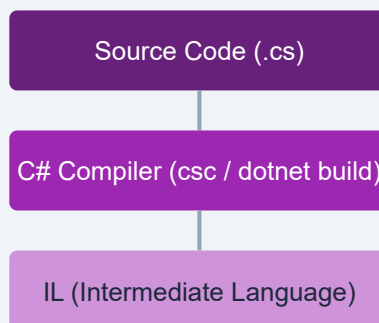
Console Output

```
Console.WriteLine("With new line");  
Console.Write("Without new line");  
Console.WriteLine($"Interpolated: {value}");
```

Comments

```
// Single-line  
/* Multi-line */  
/// XML documentation
```

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Variables & Control Flow

Variables & Data Types

C# supports both value types (int, double, bool, char, struct) and reference types (string, class, array, object).

```
int age = 25;
double price = 19.99;
string name = "Alice";
bool isActive = true;
var number = 42; // Type inferred as int
```

Type Casting

```
// Implicit (safe)
int num = 100;
double d = num;

// Explicit (may lose data)
double pi = 3.14;
int truncated = (int)pi;

// Convert class
string input = "42";
int parsed = Convert.ToInt32(input);
int parsed2 = int.Parse(input);
```

User Input

```
Console.Write("Enter your name: ");
string name = Console.ReadLine();
Console.Write("Enter your age: ");
int age = int.Parse(Console.ReadLine());
```

Operators

```
// Arithmetic: +, -, *, /, %
// Comparison: ==, !=, <, >, <=, >=
// Logical: &&, ||, !
// Assignment: =, +=, -=, *=, /=, ++, --
// Math: Math.Pow, Math.Sqrt, Math.Max, Math.Min, Math.Round
```

If/Else & Switch

```
if (age >= 18) {
    Console.WriteLine("Adult");
} else if (age >= 13) {
    Console.WriteLine("Teen");
} else {
    Console.WriteLine("Child");
}

int day = 3;
switch (day) {
    case 1: Console.WriteLine("Monday"); break;
    case 2: Console.WriteLine("Tuesday"); break;
    default: Console.WriteLine("Other"); break;
}
```

Loops

```
// For loop
for (int i = 0; i < 5; i++) {
    Console.WriteLine(i);
}

// While loop
int count = 0;
while (count < 5) {
    Console.WriteLine(count);
    count++;
}

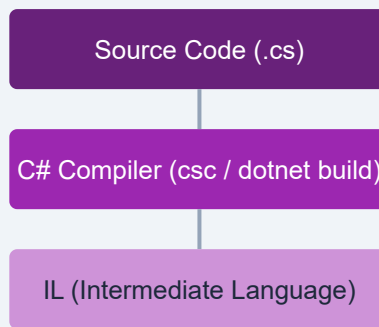
// Do-While
int x = 0;
do {
    Console.WriteLine(x);
    x++;
} while (x < 5);
```

Strings & Booleans

```
string greeting = "Hello";
Console.WriteLine(greeting.Length);
Console.WriteLine(greeting.ToUpper());
Console.WriteLine(greeting.Contains("ell"));

bool isComplete = true;
bool isGreater = 10 > 5; // true
```

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Arrays

Array Basics

Arrays store multiple values of the same type in a fixed-size collection.

```
int[] numbers = new int[5];
int[] scores = { 95, 87, 92, 78, 88 };
string[] names = new string[] { "Alice", "Bob", "Charlie" };
```

Accessing & Modifying

```
int first = scores[0];
scores[1] = 90;
int length = scores.Length;
```

Iterating

```
// For loop
for (int i = 0; i < scores.Length; i++) {
    Console.WriteLine(scores[i]);
}

// Foreach loop
foreach (int score in scores) {
    Console.WriteLine(score);
}
```

Multi-Dimensional Arrays

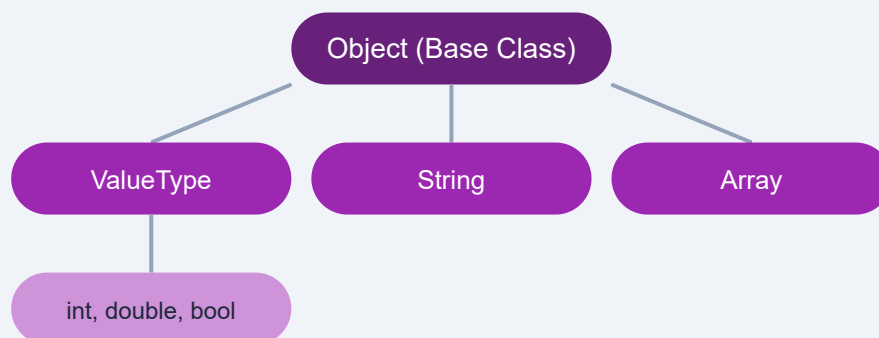
```
// Rectangular
int[,] matrix = { { 1, 2 }, { 3, 4 } };
int val = matrix[0, 1];

// Jagged (array of arrays)
int[][] jagged = new int[3][];
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 3, 4, 5 };
```

Array Class Methods

```
Array.Sort(scores);
Array.Reverse(scores);
int index = Array.IndexOf(scores, 92);
Array.Clear(scores, 0, scores.Length);
```

C# Class Hierarchy



C# is a strongly-typed, object-oriented language.
All types inherit from System.Object.

Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Methods

Defining Methods

Methods are reusable blocks of code that perform specific tasks.

```
static int Add(int a, int b) {  
    return a + b;  
}  
  
static void Greet(string name) {  
    Console.WriteLine($"Hello, {name}!");  
}  
  
// Calling methods  
int sum = Add(5, 3);  
Greet("Alice");
```

Parameters - ref, out, in

```
// ref - must be initialized before passing  
void Double(ref int num) { num *= 2; }  
int value = 5;  
Double(ref value); // value becomes 10  
  
// out - must be assigned inside the method  
void TryParse(string s, out int result) {  
    result = int.TryParse(s, out int r) ? r : 0;  
}  
  
// in - read-only reference  
void Display(in int num) {  
    Console.WriteLine(num);  
}
```

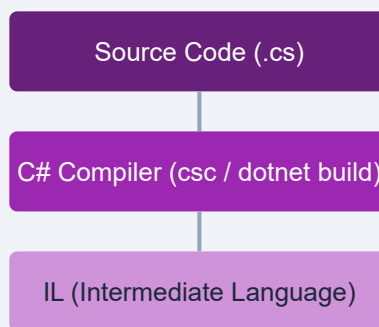
Method Overloading

```
static int Add(int a, int b) => a + b;  
static double Add(double a, double b) => a + b;  
static int Add(int a, int b, int c) => a + b + c;
```

Expression-Bodied Members

```
static int Square(int x) => x * x;  
static void Print(string msg) => Console.WriteLine(msg);
```

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

OOP Concepts

Classes & Objects

A class is a blueprint for creating objects. An object is an instance of a class.

```
class Person {  
    public string Name { get; set; }  
    public int Age { get; set; }  
  
    public Person(string name, int age) {  
        Name = name;  
        Age = age;  
    }  
  
    public void Introduce() {  
        Console.WriteLine($"Hi, I'm {Name} and I'm {Age}.");  
    }  
}  
  
// Creating objects  
Person p = new Person("Alice", 30);  
p.Introduce();
```

Access Modifiers

- `public` — accessible from anywhere
- `private` — only within the same class
- `protected` — within class and derived classes
- `internal` — within the same assembly

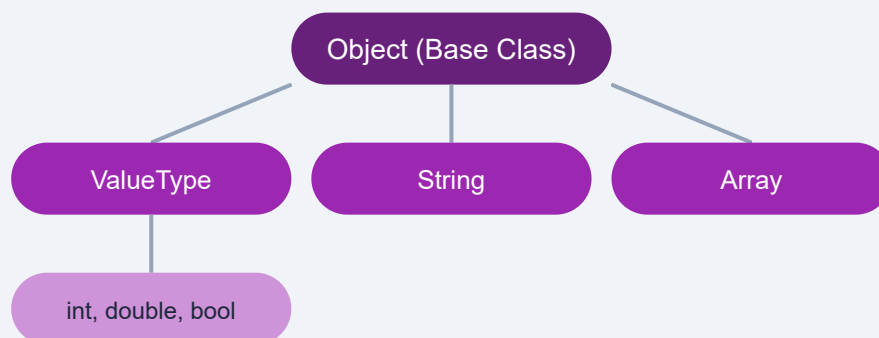
Properties

```
class Product {  
    private decimal price;  
    public decimal Price {  
        get { return price; }  
        set { if (value > 0) price = value; }  
    }  
  
    // Auto-property  
    public string Name { get; set; }  
  
    // Read-only property  
    public decimal Tax => Price * 0.08m;  
}
```

Static Members

```
class MathHelper {  
    public static double PI = 3.14159;  
    public static int Add(int a, int b) => a + b;  
}  
Console.WriteLine(MathHelper.PI);
```

C# Class Hierarchy



C# is a strongly-typed, object-oriented language.
All types inherit from System.Object.

Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Advanced OOP

Inheritance

A class can inherit members from a base class using the `:` syntax.

```
class Animal {
    public string Name { get; set; }
    public void Eat() => Console.WriteLine("Eating...");
}

class Dog : Animal {
    public void Bark() => Console.WriteLine("Woof!");
}

Dog d = new Dog();
d.Name = "Buddy";
d.Eat();
d.Bark();
```

Polymorphism (virtual/override)

```
class Shape {
    public virtual double Area() => 0;
}

class Circle : Shape {
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
}

class Rectangle : Shape {
    public double Width { get; set; }
    public double Height { get; set; }
    public override double Area() => Width * Height;
}
```

Abstract Classes

```
abstract class Vehicle {  
    public abstract void Start();  
    public void Stop() => Console.WriteLine("Stopped");  
}  
class Car : Vehicle {  
    public override void Start() => Console.WriteLine("Car started");  
}
```

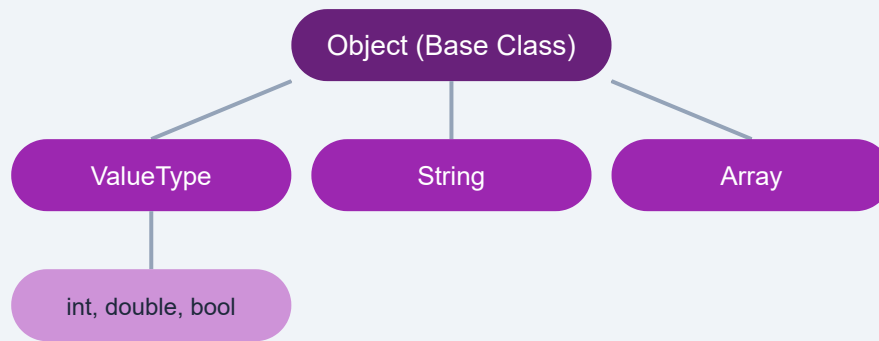
Interfaces

```
interface IComparable {  
    int CompareTo(object obj);  
}  
interface IPrintable {  
    void Print();  
}
```

Enums

```
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }  
Days today = Days.Wednesday;  
Console.WriteLine((int)today);
```

C# Class Hierarchy



C# is a strongly-typed, object-oriented language.
All types inherit from System.Object.

Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Exceptions

Try/Catch/Finally

Exceptions handle runtime errors gracefully without crashing the program.

```
try {
    int[] numbers = { 1, 2, 3 };
    Console.WriteLine(numbers[5]); // Throws IndexOutOfRangeException
} catch (IndexOutOfRangeException ex) {
    Console.WriteLine($"Error: {ex.Message}");
} catch (Exception ex) {
    Console.WriteLine($"General error: {ex.Message}");
} finally {
    Console.WriteLine("This runs whether or not an exception occurs");
}
```

Custom Exceptions

```
class AgeException : Exception {
    public AgeException(string message) : base(message) { }
}

void ValidateAge(int age) {
    if (age < 0 || age > 150) {
        throw new AgeException("Age must be between 0 and 150");
    }
}
```

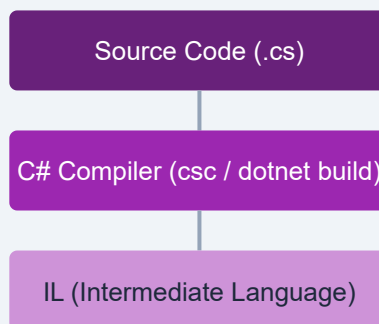
Using Statement (IDisposable)

```
using (var reader = new StreamReader("file.txt")) {  
    string content = reader.ReadToEnd();  
} // Automatically disposes the resource
```

Exception Properties

```
try { /* code */ }  
catch (Exception ex) {  
    Console.WriteLine(ex.Message);    // Human-readable description  
    Console.WriteLine(ex.StackTrace); // Call stack  
    Console.WriteLine(ex.InnerException); // Nested exception  
}
```

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

File Handling

File Class

The static `File` class provides simple file operations.

```
using System.IO;

// Reading
string content = File.ReadAllText("data.txt");
string[] lines = File.ReadAllLines("data.txt");

// Writing
File.WriteAllText("output.txt", "Hello World");
File.AppendAllText("log.txt", "New log entry\n");

// Checking & Deleting
if (File.Exists("old.txt")) {
    File.Delete("old.txt");
}

// Copy & Move
File.Copy("source.txt", "backup.txt", overwrite: true);
File.Move("temp.txt", "final.txt");
```

StreamReader & StreamWriter

```
// Reading line by line
using (StreamReader sr = new StreamReader("data.txt")) {
    string line;
    while ((line = sr.ReadLine()) != null) {
        Console.WriteLine(line);
    }
}

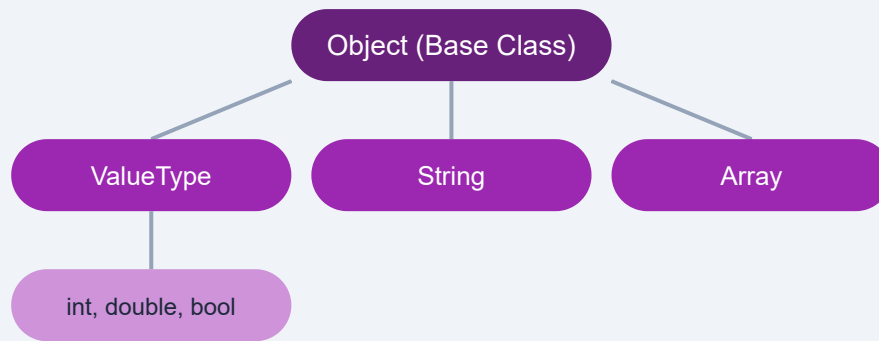
// Writing with StreamWriter
using (StreamWriter sw = new StreamWriter("output.txt")) {
    sw.WriteLine("First line");
    sw.WriteLine("Second line");
}
```

Directory & Path

```
// Directory operations
Directory.CreateDirectory("MyFolder");
string[] files = Directory.GetFiles("MyFolder");
string[] dirs = Directory.GetDirectories(".");

// Path utilities
string fullPath = Path.Combine("folder", "file.txt");
string ext = Path.GetExtension("file.txt");
string name = Path.GetFileNameWithoutExtension("file.txt");
```

C# Class Hierarchy



C# is a strongly-typed, object-oriented language.
All types inherit from System.Object.

Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Advanced Topics

LINQ (Language Integrated Query)

LINQ allows querying collections with SQL-like syntax or method chains.

```
using System.Linq;
int[] numbers = { 5, 2, 8, 1, 9, 3 };

// Query syntax
var evenQuery = from n in numbers
                where n % 2 == 0
                orderby n
                select n;

// Method syntax
var evenMethod = numbers.Where(n => n % 2 == 0)
                       .OrderBy(n => n)
                       .ToList();

// Common LINQ methods
numbers.Where(n => n > 5);
numbers.Select(n => n * 2);
numbers.OrderBy(n => n);
numbers.GroupBy(n => n % 2);
numbers.Average();
numbers.Sum();
numbers.Min(); numbers.Max();
```

Delegates

```
// Action - no return value
Action log = msg => Console.WriteLine(msg);
log("Hello from delegate");

// Func - returns a value
Func add = (a, b) => a + b;
int result = add(5, 3);
```

Generics

```
List names = new List();
names.Add("Alice");
names.Add("Bob");
names.Remove("Alice");

Dictionary scores = new();
scores["Alice"] = 95;
scores["Bob"] = 87;

Queue queue = new Queue();
queue.Enqueue("First");
string next = queue.Dequeue();

Stack stack = new Stack();
stack.Push(1);
int top = stack.Pop();
```

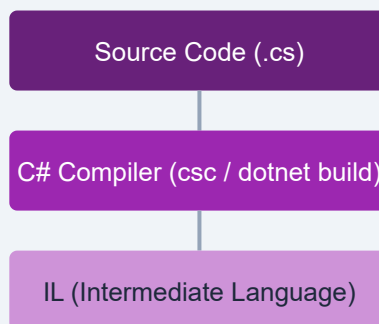
Async/Await

```
async Task FetchDataAsync() {  
    using HttpClient client = new HttpClient();  
    string result = await client.GetStringAsync("https://api.example.com");  
    return result;  
}  
  
// Call async method  
string data = await FetchDataAsync();
```

Nullable Types

```
int? maybeNumber = null;  
if (maybeNumber.HasValue) {  
    Console.WriteLine(maybeNumber.Value);  
}  
  
int actual = maybeNumber ?? 0; // Null-coalescing  
int length = name?.Length ?? 0; // Null-conditional
```

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

Practice & Resources

Practical Examples

```
// Add Two Numbers
int Add(int a, int b) => a + b;

// Reverse a String
string Reverse(string s) => new string(s.Reverse().ToArray());

// Check Palindrome
bool IsPalindrome(string s) => s == Reverse(s);

// FizzBuzz
for (int i = 1; i <= 100; i++) {
    if (i % 15 == 0) Console.WriteLine("FizzBuzz");
    else if (i % 3 == 0) Console.WriteLine("Fizz");
    else if (i % 5 == 0) Console.WriteLine("Buzz");
    else Console.WriteLine(i);
}

// Factorial
int Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);
```

Exercises

1. Write a program that counts vowels in a string
2. Create a class BankAccount with Deposit and Withdraw methods
3. Read a CSV file and calculate the average of a column
4. Build a simple to-do list app using List
5. Implement a generic method that swaps two values

Quiz Questions

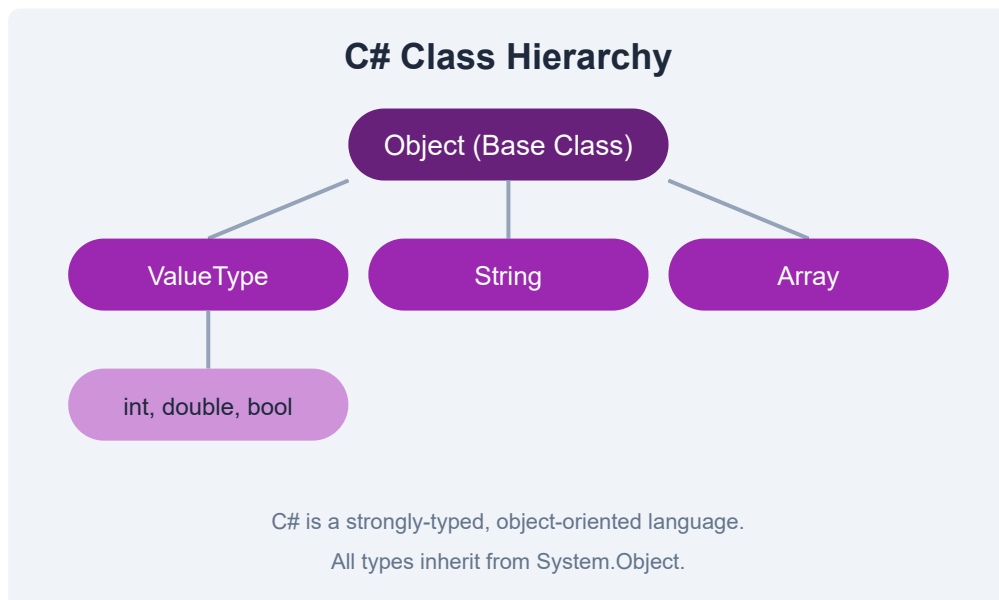
- What is the difference between ref and out parameters?
- What is the purpose of the using statement?
- What is the difference between abstract class and interface?
- What does the var keyword do?

4-Week Study Plan

- Basics, variables, control flow, arrays, methods
- OOP, classes, inheritance, polymorphism
- Exceptions, file I/O, LINQ, generics
- Async, delegates, build a mini project

Resources

- Microsoft C# Documentation (learn.microsoft.com/dotnet/csharp)
- .NET Fiddle (dotnetfiddle.net) — online compiler
- C# Yellow Book by Rob Miles



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.

References

C# Keywords Quick Reference

```
Access Modifiers: public, private, protected, internal
Types: int, double, float, decimal, char, bool, string, object, var
Loops: for, foreach, while, do
Conditionals: if, else, switch, case, default
OOP: class, struct, interface, enum, record, abstract, virtual, override, sealed
Exceptions: try, catch, finally, throw, using
Async: async, await, Task
Other: new, this, base, null, is, as, typeof, nameof, yield, partial, extern
```

Built-in Types

```
int        // 32-bit integer (-2.1B to 2.1B)
long       // 64-bit integer
float      // 32-bit floating point (7 digits precision)
double     // 64-bit floating point (15-16 digits)
decimal    // 128-bit for financial/monetary (28-29 digits)
char       // Single Unicode character
bool       // true or false
string     // Sequence of characters
object     // Base type of all types
byte       // 8-bit unsigned (0-255)
```

Common String Methods

```
Length, ToUpper(), ToLower(), Trim(), Substring(start, length),  
IndexOf(), LastIndexOf(), Contains(), StartsWith(), EndsWith(),  
Replace(old, new), Split(separator), Join(separator, array),  
ToCharArray(), PadLeft(), PadRight()
```

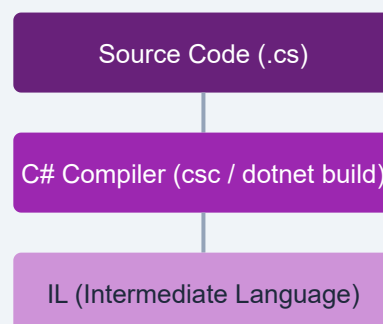
Common LINQ Operators

```
Where, Select, OrderBy, ThenBy, GroupBy, Join, Aggregate,  
Any, All, Contains, Count, Sum, Average, Min, Max,  
First, FirstOrDefault, Single, SingleOrDefault,  
Skip, Take, Distinct, Union, Intersect, Except
```

Further Reading

- docs.microsoft.com/dotnet/csharp
- learn.microsoft.com/en-us/dotnet/api
- github.com/dotnet/roslyn

C# Program Execution Flow



Practice Task

Write a C# program that applies the concepts covered in this chapter. Experiment with different inputs and test your understanding.